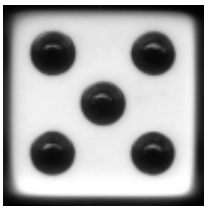
Project 1: The Game of Hog **hog.zip (hog.zip)**



I know! I'll use my
Higher-order functions to
Order higher rolls.

Introduction

Important submission note: For full credit:

- Submit with Phase 1 complete by **Tuesday, September 8** (worth 1 pt).
- Submit with all phases complete by **Friday, September 11**.

Although Phase 1 is due only a few days before the rest, you should not put off completing Phase 1. We recommend starting and finishing Phase 1 as soon as possible.

The entire project can be completed with a partner.

You can get 1 bonus point by submitting the entire project by **Thursday, September 10**.

In this project, you will develop a simulator and multiple strategies for the dice game Hog. You will need to use *control statements* and *higher-order functions* together, as described in Sections 1.2 through 1.6 of Composing Programs (http://composingprograms.com).

Rules

In Hog, two players alternate turns trying to be the first to end a turn with at least 100 total points. On each turn, the current player chooses some number of dice to roll, up to 10. That player's score for the turn is the sum of the dice outcomes. However, a player who rolls too many dice risks:

- **Pig Out**. If any of the dice outcomes is a 1, the current player's score for the turn is 1.

Examples

In a normal game of Hog, those are all the rules. To spice up the game, we'll include some special rules:

- **Free Bacon**. A player who chooses to roll zero dice scores `k+3` points, where `k` is the `n` th digit of pi after the decimal point, and `n` is the total score of their opponent. As a special case, if the opponent's score is `n = 0`, then `k = 3` (the digit of pi before the decimal point).

Examples

- **Swine Align**. After points for the turn are added to the current player's score, if both players have a positive score and the Greatest Common Divisor (GCD) of the current player's score and the opponent's score is at least 10, take another turn.

Examples

- **Pig Pass**. After points for the turn are added to the current player's score, if the current player's score is lower than the opponent's score and the difference between them is less than 3, the current player takes another turn.

Examples

Final Product

Our staff solution to the project can be played at hog.cs61a.org (https://hog.cs61a.org) -- try it out! When you finish the project, you'll have implemented a significant part of this game yourself.

Download starter files

To get started, download all of the project code as a zip archive (hog.zip). Below is a list of all the files you will see in the archive. However, you only have to make changes to `hog.py`.

- `hog.py` : A starter implementation of Hog
- `dice.py` : Functions for rolling dice
- `hog_gui.py` : A graphical user interface (GUI) for Hog
- `ucb.py` : Utility functions for CS 61A
- `ok` : CS 61A autograder
- `tests` : A directory of tests used by `ok`

- `gui_files` : A directory of various things used by the web GUI

Logistics

The project is worth 25 points. 22 points are assigned for correctness, 1 point for submitting Part I by the checkpoint date, and 2 points for the overall composition (https://cs61a.org//articles/composition.html).

You will turn in the following files:

- `hog.py`

You do not need to modify or turn in any other files to complete the project. To submit the project, run the following command:

```
python3 ok --submit
```

You will be able to view your submissions on the Ok dashboard (http://ok.cs61a.org).

For the functions that we ask you to complete, there may be some initial code that we provide. If you would rather not use that code, feel free to delete it and start from scratch. You may also add new function definitions as you see fit.

However, please do **not** modify any other functions. Doing so may result in your code failing our autograder tests. Also, please do not change any function signatures (names, argument order, or number of arguments).

Throughout this project, you should be testing the correctness of your code. It is good practice to test often, so that it is easy to isolate any problems. However, you should not be testing *too* often, to allow yourself time to think through problems.

We have provided an **autograder** called `ok` to help you with testing your code and tracking your progress. The first time you run the autograder, you will be asked to **log in with your Ok account using your web browser**. Please do so. Each time you run `ok`, it will back up your work and progress on our servers.

The primary purpose of `ok` is to test your implementations.

We recommend that you submit **after you finish each problem**. Only your last submission will be graded. It is also useful for us to have more backups of your code in case you run into a submission issue. **If you forget to submit, your last backup will be automatically converted to a submission.**

If you do not want us to record a backup of your work or information about your progress, you can run

```
python3 ok --local
```

With this option, no information will be sent to our course servers. If you want to test your code interactively, you can run

```
python3 ok -q [question number] -i
```

with the appropriate question number (e.g. `01`) inserted. This will run the tests for that question until the first one you failed, then give you a chance to test the functions you wrote interactively.

You can also use the debug printing feature in OK by writing

```
print("DEBUG:", x)
```

which will produce an output in your terminal without causing OK tests to fail with extra output.

Graphical User Interface

A **graphical user interface** (GUI, for short) is provided for you. At the moment, it doesn't work because you haven't implemented the game logic. Once you complete the `play` function, you will be able to play a fully interactive version of Hog!

Once you've done that, you can run the GUI from your terminal:

```
python3 hog_gui.py
```

The GUI is an open-source project hosted on Github (https://github.com/Cal-CS-61A-Staff/hog-gui).

Phase 1: Simulator

In the first phase, you will develop a simulator for the game of Hog.

Problem 0 (0 pt)

The `dice.py` file represents dice using non-pure zero-argument functions. These functions are non-pure because they may have different return values each time they are called. The documentation of `dice.py` describes the two different types of dice used in the project:

- Dice can be fair, meaning that they produce each possible outcome with equal probability. Example: `six_sided`.
- For testing functions that use dice, deterministic test dice always cycle through a fixed sequence of values that are passed as arguments to the `make_test_dice` function.

Before writing any code, read over the `dice.py` file and check your understanding by unlocking the following tests.

```
python3 ok -q 00 -u
```

This should display a prompt that looks like this:

```
=====================================================================
Assignment: Project 1: Hog
Ok, version v1.5.2
=====================================================================

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Unlocking tests

At each "? ", type what you would expect the output to be.
Type exit() to quit

---------------------------------------------------------------------
Question 0 > Suite 1 > Case 1
(cases remaining: 1)

>>> test_dice = make_test_dice(4, 1, 2)
>>> test_dice()
?
```

You should type in what you expect the output to be. To do so, you need to first figure out what `test_dice` will do, based on the description above.

You can exit the unlocker by typing `exit()`. **Typing Ctrl-C on Windows to exit out of the unlocker has been known to cause problems, so avoid doing so.**

Hint Video

Problem 1 (2 pt)

Implement the `roll_dice` function in `hog.py`. It takes two arguments: a positive integer called `num_rolls` giving the number of dice to roll and a `dice` function. It returns the number of points scored by rolling the dice that number of times in a turn: either the sum of the outcomes or 1 (*Pig Out*).

> The Pig Out rule:

- **Pig Out**. If any of the dice outcomes is a 1, the current player's score for the turn is 1.

Examples

To obtain a single outcome of a dice roll, call `dice()`. You should call `dice()` exactly `num_rolls` times in the body of `roll_dice`. **Remember to call `dice()` exactly `num_rolls` times even if *Pig Out* happens in the middle of rolling.** In this way, you correctly simulate rolling all the dice together.

Understand the problem:

Before writing any code, unlock the tests to verify your understanding of the question. **Note: you will not be able to test your code using OK until you unlock the test cases for the corresponding question**.

```
python3 ok -q 01 -u
```

Write code and check your work:

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 01
```

Hint Video

Debugging Tips

Problem 2 (1 pt)

Implement `free_bacon`, which takes the opponent's current `score` and returns the number of points scored by rolling 0 dice. Assume that `score` is less than 100.

> The Free Bacon rule:

- **Free Bacon**. A player who chooses to roll zero dice scores `k+3` points, where `k` is the `n` th digit of pi after the decimal point, and `n` is the total score of their opponent. As a special case, if the opponent's score is `n = 0`, then `k = 3` (the digit of pi before the decimal point).

Examples

The provided code includes the `FIRST_101_DIGITS_OF_PI` as an integer, which is stored locally as `pi`. One way to solve this problem is to trim `pi` until it has only `score+1` digits, then return 3 more than the last digit of `pi`.

Hint: An integer `n` has `k+1` digits if `n` is less than `pow(10, k+1)` but greater than or equal to `pow(10, k)`.

You are allowed to implement this function in any way you want. You don't have to follow this recommended method or use the provided starter code. However, **You may not use for loops or square brackets `[ ]` in your implementation, since we haven't covered those yet.**

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 02 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 02
```

You can also test `free_bacon` interactively by entering `python3 -i hog.py` in the terminal and then calling `free_bacon` with various inputs.

Hint Video

Problem 3 (2 pt)

Implement the `take_turn` function, which returns the number of points scored for a turn by rolling the given `dice` `num_rolls` times.

Your implementation of `take_turn` should call both `roll_dice` and `free_bacon` when possible.

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 03 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 03
```

Hint Video

Problem 4a (1 pt)

Implement `swine_align`, which takes the current player and opponent scores and returns whether the current player will take another turn due to Swine Align. **Note:** In the special case where one score is 0, return `False`.

> The Swine Align rule:

- **Swine Align**. After points for the turn are added to the current player's score, if both players have a positive score and the Greatest Common Divisor (GCD) of the current player's score and the opponent's score is at least 10, take another turn.

  Examples

> **Hint**: The expression `n % d == 0` is true if and only if `n` is a multiple of `d`.

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 04a -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 04a
```

Hint Video

Problem 4b (1 pt)

Implement `pig_pass`, which takes the current player and opponent scores and returns whether the current player will take an extra turn due to Pig Pass.

> The Pig Pass rule:

- **Pig Pass**. After points for the turn are added to the current player's score, if the current player's score is lower than the opponent's score and the difference between them is less than 3, the current player takes another turn.

Examples

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 04b -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 04b
```

Hint Video

Problem 5 (3 pt)

Implement the `play` function, which simulates a full game of Hog. Players alternate turns rolling dice until one of the players reaches the `goal` score.

To determine how many dice are rolled each turn, each player uses their respective strategy (Player 0 uses `strategy0` and Player 1 uses `strategy1`). A *strategy* is a function that, given a player's score and their opponent's score, returns the number of dice that the current player will roll in the turn. **Only call a strategy function once per turn (or risk breaking the GUI).** Don't worry about implementing strategies yet. You'll do that in Phase 3.

When the game ends, `play` returns the final total scores of both players, with Player 0's score first and Player 1's score second.

Hints:

- You should call the functions you have implemented already.
- Call `take_turn` with three arguments. Only call `take_turn` once per turn.
- Call `extra_turn` to determine if the current player will take another turn, either due to Swine Align or Pig Pass.
- You can get the number of the other player (either 0 or 1) by calling the provided function `other`.
- You can ignore the `say` argument to the `play` function for now. You will use it in Phase 2 of the project.

Rules Clarification: A player can take more than two consecutive turns. For example, if the score after their first turn is 10 vs 20, they go again due to Swine Align. If they score 8 points and now the score is 18 vs 20, they go a third time due to Pig Pass. If they score 12 points and now the score is 30 vs 20, they go a fourth time in a row.

Note: It's not mathematically possible for Swine Align and Pig Pass to be active simultaneously.

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 05 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 05
```

Once you are finished, you will be able to play a graphical version of the game. We have provided a file called `hog_gui.py` that you can run from the terminal:

```
python3 hog_gui.py
```

The GUI relies on your implementation, so if you have any bugs in your code, they will be reflected in the GUI. This means you can also use the GUI as a debugging tool; however, it's better to run the tests first.

Note: The hog gui now works! To get an updated version, re-download the zip file from this webpage and drag over the new copies of `hog_gui.py` and `gui_files/`.

Make sure to submit your work so far before the checkpoint deadline:

```
python3 ok --submit
```

Check to make sure that you did all the problems in Phase 1:

```
python3 ok --score
```

Hint Video

Congratulations! You have finished Phase 1 of this project!

Phase 2: Commentary

In the second phase, you will implement commentary functions that print remarks about the game after each turn, such as, `"22 points! That's the biggest gain yet for Player 1."`

A commentary function takes two arguments, Player 0's current score and Player 1's current score. It can print out commentary based on either or both current scores and any other information in its parent environment. Since commentary can differ from turn to turn

depending on the current point situation in the game, a commentary function always returns another commentary function to be called on the next turn. The only side effect of a commentary function should be to print.

Commentary examples

The function `say_scores` in `hog.py` is an example of a commentary function that simply announces both players' scores. Note that `say_scores` returns itself, meaning that the same commentary function will be called each turn.

```
def say_scores(score0, score1):
    """A commentary function that announces the score for each player."""
    print("Player 0 now has", score0, "and Player 1 now has", score1)
    return say_scores
```

The function `announce_lead_changes` is an example of a higher-order function that returns a commentary function that tracks lead changes. A different commentary function will be called each turn.

```
def announce_lead_changes(last_leader=None):
    """Return a commentary function that announces lead changes.

    >>> f0 = announce_lead_changes()
    >>> f1 = f0(5, 0)
    Player 0 takes the lead by 5
    >>> f2 = f1(5, 12)
    Player 1 takes the lead by 7
    >>> f3 = f2(8, 12)
    >>> f4 = f3(8, 13)
    >>> f5 = f4(15, 13)
    Player 0 takes the lead by 2
    """
    def say(score0, score1):
        if score0 > score1:
            leader = 0
        elif score1 > score0:
            leader = 1
        else:
            leader = None
        if leader != None and leader != last_leader:
            print('Player', leader, 'takes the lead by', abs(score0 - score1))
        return announce_lead_changes(leader)
    return say
```

You should also understand the function `both`, which takes two commentary functions (`f` and `g`) and returns a *new* commentary function. This returned commentary function returns *another* commentary function which calls the functions returned by calling `f` and `g`, in that order.

```
def both(f, g):
    """Return a commentary function that says what f says, then what g says.

    NOTE: the following game is not possible under the rules, it's just
    an example for the sake of the doctest

    >>> h0 = both(say_scores, announce_lead_changes())
    >>> h1 = h0(10, 0)
    Player 0 now has 10 and Player 1 now has 0
    Player 0 takes the lead by 10
    >>> h2 = h1(10, 8)
    Player 0 now has 10 and Player 1 now has 8
    >>> h3 = h2(10, 17)
    Player 0 now has 10 and Player 1 now has 17
    Player 1 takes the lead by 7
    """
    def say(score0, score1):
        return both(f(score0, score1), g(score0, score1))
    return say
```

Hint Video

Problem 6 (2 pt)

Update your `play` function so that a commentary function is called at the end of each turn. The return value of calling a commentary function gives you the commentary function to call on the next turn.

For example, `say(score0, score1)` should be called at the end of the first turn. Its return value (another commentary function) should be called at the end of the second turn. Each consecutive turn, call the function that was returned by the call to the previous turn's commentary function.

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 06 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 06
```

Problem 7 (3 pt)

Implement the `announce_highest` function, which is a higher-order function that returns a commentary function. This commentary function announces whenever a particular player gains more points in a turn than ever before. E.g., `announce_highest(1)` ignores Player 0 entirely and just print information about Player 1. (So does its return value; another commentary function about only Player 1.) To compute the gain, it must compare the score from last turn (`last_score`) to the score from this turn for the player of interest, which is designated by the `who` argument. This function must also keep track of the highest gain for the player so far, which is store as `running_high` .

The way in which `announce_highest` announces is very specific, and your implementation should match the doctests provided. Don't worry about singular versus plural when announcing point gains; you should simply use "point(s)" for both cases.

> **Hint:** The `announce_lead_changes` function provided to you is an example of how to keep track of information using commentary functions. If you are stuck, first make sure you understand how `announce_lead_changes` works.

> **Hint.** If you're getting a `local variable [var] reference before assignment` error:
>
> This happens because in Python, you aren't normally allowed to modify variables defined in parent frames. Instead of reassigning `[var]` , the interpreter thinks you're trying to define a new variable within the current frame. We'll learn about how to work around this in a future lecture, but it is not required for this problem.
>
> To fix this, you have two options:
>
> 1) Rather than reassigning `[var]` to its new value, create a new variable to hold that new value. Use that new variable in future calculations.
>
> 2) For this problem specifically, avoid this issue entirely by not using assignment statements at all. Instead, pass new values in as arguments to a call to `announce_highest` .

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 07 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 07
```

When you are done, you will see commentary in the GUI:

```
python3 hog_gui.py
```

The commentary in the GUI is generated by passing the following function as the `say` argument to `play`.

```
both(announce_highest(0), both(announce_highest(1), announce_lead_changes()))
```

Great work! You just finished Phase 2 of the project!

Phase 3: Strategies

In the third phase, you will experiment with ways to improve upon the basic strategy of always rolling a fixed number of dice. First, you need to develop some tools to evaluate strategies.

Hint Video

Problem 8 (2 pt)

Implement the `make_averaged` function, which is a higher-order function that takes a function `original_function` as an argument. It returns another function that takes the same number of arguments as `original_function` (the function originally passed into `make_averaged`). This returned function differs from the input function in that it returns the average value of repeatedly calling `original_function` on the same arguments. This function should call `original_function` a total of `trials_count` times and return the average of the results.

To implement this function, you need a new piece of Python syntax! You must write a function that accepts an arbitrary number of arguments, then calls another function using exactly those arguments. Here's how it works.

Instead of listing formal parameters for a function, you can write `*args`. To call another function using exactly those arguments, you call it again with `*args`. For example,

```
>>> def printed(f):
...     def print_and_return(*args):
...         result = f(*args)
...         print('Result:', result)
...         return result
...     return print_and_return
>>> printed_pow = printed(pow)
>>> printed_pow(2, 8)
Result: 256
256
>>> printed_abs = printed(abs)
>>> printed_abs(-10)
Result: 10
10
```

Read the docstring for `make_averaged` carefully to understand how it is meant to work.

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 08 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 08
```

Problem 9 (2 pt)

Implement the `max_scoring_num_rolls` function, which runs an experiment to determine the number of rolls (from 1 to 10) that gives the maximum average score for a turn. Your implementation should use `make_averaged` and `roll_dice`.

If two numbers of rolls are tied for the maximum average score, return the lower number. For example, if both 3 and 6 achieve a maximum average score, return 3.

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 09 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 09
```

To run this experiment on randomized dice, call `run_experiments` using the `-r` option:

```
python3 hog.py -r
```

Running experiments For the remainder of this project, you can change the implementation of `run_experiments` as you wish. By calling `average_win_rate`, you can evaluate various Hog strategies. For example, change the first `if False:` to `if True:` in order to evaluate `always_roll(8)` against the baseline strategy of `always_roll(6)`.

Some of the experiments may take up to a minute to run. You can always reduce the number of trials in your call to `make_averaged` to speed up experiments.

Running experiments won't affect your score on the project.

Problem 10 (1 pt)

A strategy can try to take advantage of the *Free Bacon* rule by rolling 0 when it is most beneficial to do so. Implement `bacon_strategy`, which returns 0 whenever rolling 0 would give **at least** `cutoff` points and returns `num_rolls` otherwise.

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 10 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 10
```

Once you have implemented this strategy, change `run_experiments` to evaluate your new strategy against the baseline. Is it better than just rolling 4?

Problem 11 (2 pt)

A strategy can also take advantage of the *Pig Pass* and *Swine Align* rules. The extra turn strategy always rolls 0 if doing so triggers an extra turn. In other cases, it rolls 0 if rolling 0 would give **at least** `cutoff` points. Otherwise, the strategy rolls `num_rolls`.

> **Hint**: You can use the function `bacon_strategy` you defined in Problem 10

Before writing any code, unlock the tests to verify your understanding of the question.

```
python3 ok -q 11 -u
```

Once you are done unlocking, begin implementing your solution. You can check your correctness with:

```
python3 ok -q 11
```

Once you have implemented this strategy, update `run_experiments` to evaluate your new strategy against the baseline. You should find that it gives a significant edge over `always_roll(4)`.

Optional: Problem 12 (0 pt)

Implement `final_strategy`, which combines these ideas and any other ideas you have to achieve a high win rate against the `always_roll(4)` strategy. Some suggestions:

- `extra_turn_strategy` is a good default strategy to start with.
- There's no point in scoring more than 100. Check whether you can win by rolling 0, 1 or 2 dice. If you are in the lead, you might take fewer risks.
- Try to force extra turns.
- Choose the `num_rolls` and `cutoff` arguments carefully.
- Take the action that is most likely to win the game.

You can check that your final strategy is valid by running Ok.

```
python3 ok -q 12
```

> Note: `calc.py` does not currently work. We're fixing a bug with the server and will make it work soon.

You will also eventually be able to check your exact final win rate by running

```
python3 calc.py
```

This should pop up a window asking for you to confirm your identity, and then it will print out a win rate for your final strategy.

At this point, run the entire autograder to see if there are any tests that don't pass.

```
python3 ok
```

Once you are satisfied, submit to Ok to complete the project.

```
python3 ok --submit
```

If you have a partner, make sure to add them to the submission on okpy.org.

Check to make sure that you did all the problems by running

```
python3 ok --score
```

You can also play against your final strategy with the graphical user interface:

```
python3 hog_gui.py
```

The GUI will alternate which player is controlled by you.

Congratulations, you have reached the end of your first CS 61A project! If you haven't already, relax and enjoy a few games of Hog with a friend.

Hog Strategy Contest

If you're interested, you can take your implementation of Hog one step further by participating in the Hog Contest, where you play your `final_strategy` against those of other students. The winning strategies will receive extra credit and will be recognized in future semesters!

To see more, read the contest description (/~cs61a/fa20/proj/hog_contest). Or simply check out the leaderboard (https://hog-contest.cs61a.org).

CS 61A (/~cs61a/fa20/)

Weekly Schedule (/~cs61a/fa20/weekly.html)

Office Hours (/~cs61a/fa20/office-hours.html)

Staff (/~cs61a/fa20/staff.html)

Resources (/~cs61a/fa20/resources.html)

Studying Guide (/~cs61a/fa20/articles/studying.html)

Debugging Guide (/~cs61a/fa20/articles/debugging.html)

Composition Guide (/~cs61a/fa20/articles/composition.html)

Policies (/~cs61a/fa20/articles/about.html)

Assignments (/~cs61a/fa20/articles/about.html#assignments)

Exams (/~cs61a/fa20/articles/about.html#exams)

Grading (/~cs61a/fa20/articles/about.html#grading)